

**Moscow Aviation Institute
(National Research University)**

Faculty of Information Technologies and Applied Mathematics

Department of Computational Mathematics and Programming

Laboratory work No. 2, course “Discrete Analysis”

Student: M. A. Khorokhorin
Teacher: A. A. Kukhtichev
Group: M8O-206BV-24
Date: 10.05.2026
Grade:
Signature:

Moscow, 2026

Laboratory work No. 2

Task: You need to create a program library implementing the specified data structure, on the basis of which to develop a dictionary program. In the dictionary, each key, which is a case-insensitive sequence of letters of the English alphabet of no more than 256 characters, is associated with a number from 0 to $2^{64} - 1$. Different words can be assigned the same number.

Data structure variant: PATRICIA.

Time limit	15 seconds
Memory limit	512 MB
Input	standard input or <code>input.txt</code>
Output	standard output or <code>output.txt</code>

The program must process lines of the input file until its end. Each line can have the following format:

- **+ word 34** — add the word “word” with number 34 to the dictionary. The program must output “OK” if the operation was successful, “Exist” if the word is already in the dictionary.
- **- word** — remove the word “word” from the dictionary. The program must output “OK” if the word existed and was removed, “NoSuchWord” if the word was not found in the dictionary.
- **word** — find the word “word” in the dictionary. The program must output “OK: 34” if the word was found (the number following “OK:” is the number assigned to the word when it was added). If the word was not found in the dictionary, output “NoSuchWord”.
- **! Save /path/to/file** — save the dictionary to a file in binary compact representation, at the path specified by the command parameter. If successful, the program must output “OK”; if the operation fails, output a description of the error.
- **! Load /path/to/file** — load the dictionary from a file. It is assumed that the file was previously prepared using the Save command. If successful, the program must output “OK” and the loaded dictionary replaces the current one; if unsuccessful, diagnostics must be output and the working dictionary must remain unchanged. In addition to system errors, the program must correctly handle cases where the format of the specified file does not match the dictionary data representation in the external file.

For all operations, if a system error occurs (out of memory, no write permissions, etc.), the program must output a line starting with “ERROR:” and describing the error in English. The difference between variants lies only in the data structures used: **PATRICIA**.

Input format

Each non-empty line of the input contains one of the commands described above.

Output format

For each input line, the program outputs one response line as specified above.

Example

Input:

```
+ a 1
+ A 2
+ aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa 18446744073709551615
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
A
- A
a
```

Output:

```
OK
Exist
OK
OK: 18446744073709551615
OK: 1
OK
NoSuchWord
```

1 Description

The task is to implement a dictionary based on the PATRICIA tree (Practical Algorithm To Retrieve Information Coded In Alphanumeric).

A PATRICIA tree is a compressed binary trie in which every node stores the index of the critical bit of the key that is used for branching. This avoids chains of single-child nodes and keeps the height of the tree at $O(\log n)$ on average for random keys.

The key feature of the structure is *back edges*: when during a descent the critical bit index of the next node does not exceed the index of the current node, the descent stops. The target node of such an edge is the candidate for a full string comparison.

To handle boundary cases correctly, a header node (`header_`) with a zero critical bit is used; its left child is the root of the tree. All keys are converted to lower case, which makes the dictionary case-insensitive. The dictionary supports the operations `insert`, `find`, `remove`, as well as binary `save` and `load`.

2 Source code

1 Helper functions

The function `extractBit` extracts the i -th bit of a string (numbering starts at 1, the most significant bit of the first byte is bit 1). The function `firstDifferentBit` finds the position of the first bit in which two keys differ; the result is used during insertion to determine the critical bit of the new node. The functions `writeUnsigned` and `readUnsigned` write and read unsigned integers of a given width in little-endian byte order. The public free function `isValidKey` checks that a key consists only of English letters and has length from 1 to 256, and `normalizeKey` converts a key to lower case.

2 Node structure and the trie class

Every node stores a key, a value, the critical bit index and pointers to two children. The field `size_` of the class `PatriciaTrie` tracks the number of elements. The node type is a private nested structure `PatriciaTrie::Node`; the class owns its nodes, so copying is disabled and only moving is allowed.

include/patricia_trie.hpp	
<code>class PatriciaError</code>	Exception derived from <code>std::runtime_error</code> , thrown by file operations
<code>bool isValidKey(std::string_view)</code>	Checks that a key consists of 1 to 256 English letters
<code>std::string normalizeKey(std::string_view)</code>	Converts a key to lower case
<code>class PatriciaTrie</code>	Dictionary: <code>find</code> , <code>insert</code> , <code>remove</code> , <code>size</code> , <code>empty</code> , <code>clear</code> , <code>swap</code> , <code>save</code> , <code>load</code> , <code>saveToFile</code> , <code>loadFromFile</code>
<code>struct PatriciaTrie::Node</code>	Private node: key, value, critical bit, pointers to the children

3 Search

The method `searchNode` descends the tree, choosing the branch by the critical bit, until it meets a back edge (the bit index of the next node is $\leq$ the index of the current one). The node found is a candidate for which `find` performs a full string comparison and returns the value wrapped in `std::optional`, or `std::nullopt` if the keys differ.

Methods of PatriciaTrie	
<code>std::optional<std::uint64_t> find(std::string_view key) const</code>	Returns the value of a key or <code>std::nullopt</code>
<code>bool insert(std::string_view key, std::uint64_t value)</code>	Inserts a key; returns <code>false</code> if it already exists
<code>bool remove(std::string_view key)</code>	Removes a key; returns <code>false</code> if it is absent
<code>void save(std::ostream&) const</code>	Writes the dictionary in the binary format
<code>static PatriciaTrie load(std::istream&)</code>	Reads and validates a dictionary from a stream
<code>void saveToFile(const std::string&) const</code>	Saves the dictionary to a file
<code>void loadFromFile(const std::string&)</code>	Loads a dictionary from a file and replaces the current one
<code>searchNode, findParentLink, removeNodeWithSelfLoop, removeNodeWithoutSelfLoop, saveSubtree, destroySubtree</code>	Private helper methods

4 Insertion

During insertion `searchNode` is executed first to detect a collision or to obtain a reference key. The function `firstDifferentBit` computes the critical bit of the new node. Then a second descent is performed down to the position where the new node must be inserted, after which the node is attached with a back edge to itself.

5 Removal

The method `remove` handles two cases. If the node being removed has a back edge to itself (a leaf node), `removeNodeWithSelfLoop` switches the parent link, found by `findParentLink`, to the other child. Otherwise the back edge to the node comes from another node (the holder): `removeNodeWithoutSelfLoop` moves the data of the holder into the node being removed, updates all links that pointed to the holder and physically deletes the holder.

6 Serialization

The file starts with a header: the 8-byte magic string `PATRICIA`, a 4-byte version number (currently 1) and an 8-byte number of entries. The method `saveSubtree` traverses the

tree recursively and writes every node as a triple: key length (2 bytes), the key itself and the value (8 bytes). All integers are stored in little-endian byte order. Two bytes are used for the length because by the statement a key may be 256 characters long, and one byte is not enough for this value.

The method `load` checks the magic string and the version, reads exactly the declared number of entries, validates every key (length, English letters only, lower case, no duplicates) and requires that no extra bytes follow the last entry. Entries are inserted into a new tree through `insert`. The method `loadFromFile` replaces the current state with the loaded tree through `swap` only after the whole file has been read successfully. Any error is reported with a `PatriciaError` exception: “cannot open file for writing”, “cannot open file for reading”, “failed to write file”, “invalid file format”, “unsupported file version”. In this case the working dictionary remains unchanged. The method `destroySubtree` frees the nodes recursively in the destructor and in `clear`.

7 Command processing

The class `Dictionary` parses one input line in the method `execute`: the line is split into tokens, the command is determined by the first token, keys are checked with `isValidKey` and normalized with `normalizeKey` before being passed to the tree. The responses are OK, Exist, NoSuchWord and OK: N. Errors are reported with the lines `ERROR: invalid command`, `ERROR: unknown command`, `ERROR: invalid key`, `ERROR: invalid value`, and for file commands with the text of the `PatriciaError` exception prefixed by `ERROR:.` The function `runDictionary` reads commands line by line and prints a response for every non-empty line; if memory allocation fails, it prints `ERROR: out of memory`. The function `main` only calls it for the standard streams.

include/dictionary.hpp	
<code>class Dictionary</code>	Command interpreter over <code>PatriciaTrie</code>
<code>std::optional<std::string></code> <code>execute(std::string_view line)</code>	Executes one command line and returns the response, or <code>std::nullopt</code> for an empty line
<code>void runDictionary(std::istream&, std::ostream&)</code>	Processes all input lines

8 Listings

Listing 1: include/patricia_trie.hpp

```

1 | #pragma once
2 |

```

```

3  #include <cstddef>
4  #include <cstdint>
5  #include <iosfwd>
6  #include <optional>
7  #include <stdexcept>
8  #include <string>
9  #include <string_view>
10
11 inline constexpr std::size_t MAX_KEY_LENGTH = 256;
12 inline constexpr std::uint32_t FILE_VERSION = 1;
13
14 class PatriciaError : public std::runtime_error {
15 public:
16     using std::runtime_error::runtime_error;
17 };
18
19 bool isValidKey(std::string_view key);
20 std::string normalizeKey(std::string_view key);
21
22 class PatriciaTrie {
23 public:
24     PatriciaTrie() = default;
25     ~PatriciaTrie();
26
27     PatriciaTrie(const PatriciaTrie&) = delete;
28     PatriciaTrie& operator=(const PatriciaTrie&) = delete;
29     PatriciaTrie(PatriciaTrie&& other) noexcept;
30     PatriciaTrie& operator=(PatriciaTrie&& other) noexcept;
31
32     [[nodiscard]] std::optional<std::uint64_t> find(std::string_view key) const;
33     bool insert(std::string_view key, std::uint64_t value);
34     bool remove(std::string_view key);
35
36     [[nodiscard]] std::size_t size() const { return size_; }
37     [[nodiscard]] bool empty() const { return size_ == 0; }
38     void clear();
39     void swap(PatriciaTrie& other) noexcept;
40
41     void save(std::ostream& output) const;
42     static PatriciaTrie load(std::istream& input);
43     void saveToFile(const std::string& path) const;
44     void loadFromFile(const std::string& path);
45
46 private:
47     struct Node {
48         std::string key;
49         std::uint64_t value = 0;
50         std::size_t checkBit = 0;
51         Node* left = nullptr;
52         Node* right = nullptr;
53     };
54
55     [[nodiscard]] Node* searchNode(std::string_view key) const;
56     [[nodiscard]] Node** findParentLink(const Node* target, std::string_view key) const;
57     void removeNodeWithSelfLoop(Node* target, std::string_view key);
58     void removeNodeWithoutSelfLoop(Node* target, std::string_view key);
59     static void saveSubtree(std::ostream& output, const Node* node);
60     static void destroySubtree(Node* node);
61
62     Node* header_ = nullptr;
63     std::size_t size_ = 0;
64 };

```

Listing 2: src/patricia_trie.cpp

```

1  #include "patricia_trie.hpp"
2
3  #include <algorithm>
4  #include <array>
5  #include <cstdint>
6  #include <cstring>
7  #include <fstream>
8  #include <istream>
9  #include <optional>
10 #include <ostream>
11 #include <string>
12 #include <string_view>
13 #include <utility>
14
15 namespace {
16
17 constexpr std::size_t BITS_PER_BYTE = 8;
18 constexpr unsigned HIGH_BIT = 0x80U;
19 constexpr unsigned BYTE_MASK = 0xFFU;
20 constexpr std::array<char, 8> FILE_MAGIC = {'P', 'A', 'T', 'R', 'I', 'C', 'I', 'A'};
21
22 bool extractBit(std::string_view key, std::size_t position) {
23     const std::size_t byteIndex = (position - 1) / BITS_PER_BYTE;
24     const std::size_t bitOffset = BITS_PER_BYTE - 1 - (position - 1) % BITS_PER_BYTE;
25     if (byteIndex >= key.size()) {
26         return false;
27     }
28     return ((static_cast<unsigned char>(key[byteIndex]) >> bitOffset) & 1U) != 0;
29 }
30
31 std::size_t firstDifferentBit(std::string_view lhs, std::string_view rhs) {
32     const std::size_t maxLength = std::max(lhs.size(), rhs.size());
33     for (std::size_t i = 0; i < maxLength; ++i) {
34         const unsigned left = i < lhs.size() ? static_cast<unsigned char>(lhs[i]) : 0U;
35         const unsigned right = i < rhs.size() ? static_cast<unsigned char>(rhs[i]) : 0U;
36         if (left != right) {
37             unsigned diff = left ^ right;
38             std::size_t shift = 0;
39             while ((diff & HIGH_BIT) == 0) {
40                 diff <<= 1U;
41                 ++shift;
42             }
43             return i * BITS_PER_BYTE + shift + 1;
44         }
45     }
46     return maxLength * BITS_PER_BYTE + 1;
47 }
48
49 void writeUnsigned(std::ostream& output, std::uint64_t value, std::size_t bytes) {
50     for (std::size_t i = 0; i < bytes; ++i) {
51         output.put(static_cast<char>((value >> (i * BITS_PER_BYTE)) & BYTE_MASK));
52     }
53 }
54
55 std::uint64_t readUnsigned(std::istream& input, std::size_t bytes) {
56     std::uint64_t value = 0;
57     for (std::size_t i = 0; i < bytes; ++i) {
58         const int symbol = input.get();
59         if (symbol == std::char_traits<char>::eof()) {

```

```

60         throw PatriciaError("invalid file format");
61     }
62     value |= static_cast<std::uint64_t>(static_cast<unsigned char>(symbol))
63         << (i * BITS_PER_BYTE);
64 }
65 return value;
66 }
67
68 }
69
70 bool isValidKey(std::string_view key) {
71     return !key.empty() && key.size() <= MAX_KEY_LENGTH &&
72         std::ranges::all_of(key, [](char symbol) {
73             return (symbol >= 'a' && symbol <= 'z') || (symbol >= 'A' && symbol <= 'Z');
74         });
75 }
76
77 std::string normalizeKey(std::string_view key) {
78     std::string result(key);
79     for (char& symbol : result) {
80         if (symbol >= 'A' && symbol <= 'Z') {
81             symbol = static_cast<char>(symbol - 'A' + 'a');
82         }
83     }
84     return result;
85 }
86
87 PatriciaTrie::~PatriciaTrie() {
88     clear();
89 }
90
91 PatriciaTrie::PatriciaTrie(PatriciaTrie&& other) noexcept
92     : header_(std::exchange(other.header_, nullptr)), size_(std::exchange(other.size_, 0)) {
93 }
94
95 PatriciaTrie& PatriciaTrie::operator=(PatriciaTrie&& other) noexcept {
96     if (this != &other) {
97         clear();
98         header_ = std::exchange(other.header_, nullptr);
99         size_ = std::exchange(other.size_, 0);
100     }
101     return *this;
102 }
103
104 void PatriciaTrie::clear() {
105     if (header_ != nullptr) {
106         destroySubtree(header_);
107     }
108     header_ = nullptr;
109     size_ = 0;
110 }
111
112 void PatriciaTrie::swap(PatriciaTrie& other) noexcept {
113     std::swap(header_, other.header_);
114     std::swap(size_, other.size_);
115 }
116
117 PatriciaTrie::Node* PatriciaTrie::searchNode(std::string_view key) const {
118     const Node* previous = header_;
119     Node* current = header_>left;
120     while (current->checkBit > previous->checkBit) {
121         previous = current;

```

```

122     current = extractBit(key, current->checkBit) ? current->right : current->left;
123 }
124 return current;
125 }
126
127 std::optional<std::uint64_t> PatriciaTrie::find(std::string_view key) const {
128     if (header_ == nullptr) {
129         return std::nullopt;
130     }
131     const Node* found = searchNode(key);
132     if (found->key != key) {
133         return std::nullopt;
134     }
135     return found->value;
136 }
137
138 bool PatriciaTrie::insert(std::string_view key, std::uint64_t value) {
139     if (header_ == nullptr) {
140         header_ = new Node{.key = std::string(key), .value = value};
141         header_->left = header_;
142         size_ = 1;
143         return true;
144     }
145
146     const Node* existing = searchNode(key);
147     if (existing->key == key) {
148         return false;
149     }
150
151     const std::size_t diffBit = firstDifferentBit(key, existing->key);
152
153     Node* previous = header_;
154     Node* current = header_->left;
155     while (current->checkBit > previous->checkBit && current->checkBit < diffBit) {
156         previous = current;
157         current = extractBit(key, current->checkBit) ? current->right : current->left;
158     }
159
160     Node* node = new Node{.key = std::string(key), .value = value, .checkBit = diffBit};
161     if (extractBit(key, diffBit)) {
162         node->right = node;
163         node->left = current;
164     } else {
165         node->left = node;
166         node->right = current;
167     }
168
169     if (previous == header_) {
170         header_->left = node;
171     } else if (extractBit(key, previous->checkBit)) {
172         previous->right = node;
173     } else {
174         previous->left = node;
175     }
176
177     ++size_;
178     return true;
179 }
180
181 PatriciaTrie::Node** PatriciaTrie::findParentLink(const Node* target, std::string_view key) const {
182     Node** link = &header_->left;
183     const Node* parent = header_;

```

```

184     Node* current = header_>left;
185     while (current->checkBit > parent->checkBit && current != target) {
186         parent = current;
187         link = extractBit(key, current->checkBit) ? &current->right : &current->left;
188         current = *link;
189     }
190     return link;
191 }
192
193 void PatriciaTrie::removeNodeWithSelfLoop(Node* target, std::string_view key) {
194     if (target == header_) {
195         delete header_;
196         header_ = nullptr;
197         return;
198     }
199     Node* other = target->left == target ? target->right : target->left;
200     *findParentLink(target, key) = other;
201     delete target;
202 }
203
204 void PatriciaTrie::removeNodeWithoutSelfLoop(Node* target, std::string_view key) {
205     Node* grandParent = header_;
206     Node* holder = header_;
207     Node* current = header_>left;
208     while (current->checkBit > holder->checkBit) {
209         grandParent = holder;
210         holder = current;
211         current = extractBit(key, current->checkBit) ? current->right : current->left;
212     }
213
214     const bool holderHasSelfLoop = holder->left == holder || holder->right == holder;
215     Node* holderChild = nullptr;
216     if (holderHasSelfLoop) {
217         holderChild = target;
218     } else {
219         holderChild = holder->left == target ? holder->right : holder->left;
220     }
221
222     Node* source = header_;
223     current = header_>left;
224     while (current->checkBit > source->checkBit) {
225         source = current;
226         current = extractBit(holder->key, current->checkBit) ? current->right : current->left;
227     }
228
229     target->key = std::move(holder->key);
230     target->value = holder->value;
231
232     if (source->left == holder) {
233         source->left = target;
234     }
235     if (source->right == holder) {
236         source->right = target;
237     }
238     if (target->left == holder) {
239         target->left = holderChild;
240     }
241     if (target->right == holder) {
242         target->right = holderChild;
243     }
244     if (grandParent->left == holder) {
245         grandParent->left = holderChild;

```

```

246     } else if (grandParent->right == holder) {
247         grandParent->right = holderChild;
248     }
249
250     delete holder;
251 }
252
253 bool PatriciaTrie::remove(std::string_view key) {
254     if (header_ == nullptr) {
255         return false;
256     }
257     Node* target = searchNode(key);
258     if (target->key != key) {
259         return false;
260     }
261     if (target->left == target || target->right == target) {
262         removeNodeWithSelfLoop(target, key);
263     } else {
264         removeNodeWithoutSelfLoop(target, key);
265     }
266     --size_;
267     return true;
268 }
269
270 void PatriciaTrie::saveSubtree(std::ostream& output, const Node* node) {
271     writeUnsigned(output, node->key.size(), sizeof(std::uint16_t));
272     output.write(node->key.data(), static_cast<std::streamsize>(node->key.size()));
273     writeUnsigned(output, node->value, sizeof(std::uint64_t));
274     if (node->left != nullptr && node->left->checkBit > node->checkBit) {
275         saveSubtree(output, node->left);
276     }
277     if (node->right != nullptr && node->right->checkBit > node->checkBit) {
278         saveSubtree(output, node->right);
279     }
280 }
281
282 void PatriciaTrie::destroySubtree(Node* node) {
283     if (node->left != nullptr && node->left->checkBit > node->checkBit) {
284         destroySubtree(node->left);
285     }
286     if (node->right != nullptr && node->right->checkBit > node->checkBit) {
287         destroySubtree(node->right);
288     }
289     delete node;
290 }
291
292 void PatriciaTrie::save(std::ostream& output) const {
293     output.write(FILE_MAGIC.data(), static_cast<std::streamsize>(FILE_MAGIC.size()));
294     writeUnsigned(output, FILE_VERSION, sizeof(std::uint32_t));
295     writeUnsigned(output, size_, sizeof(std::uint64_t));
296     if (header_ != nullptr) {
297         saveSubtree(output, header_);
298     }
299     output.flush();
300     if (!output.good()) {
301         throw PatriciaError("failed to write file");
302     }
303 }
304
305 PatriciaTrie PatriciaTrie::load(std::istream& input) {
306     std::array<char, FILE_MAGIC.size()> magic{};
307     if (!input.read(magic.data(), static_cast<std::streamsize>(magic.size())) ||

```

```

308     magic != FILE_MAGIC) {
309         throw PatriciaError("invalid file format");
310     }
311     if (readUnsigned(input, sizeof(std::uint32_t)) != FILE_VERSION) {
312         throw PatriciaError("unsupported file version");
313     }
314     const std::uint64_t count = readUnsigned(input, sizeof(std::uint64_t));
315
316     PatriciaTrie trie;
317     for (std::uint64_t i = 0; i < count; ++i) {
318         const std::uint64_t length = readUnsigned(input, sizeof(std::uint16_t));
319         if (length == 0 || length > MAX_KEY_LENGTH) {
320             throw PatriciaError("invalid file format");
321         }
322         std::string key(length, '\0');
323         if (!input.read(key.data(), static_cast<std::streamsize>(length)) || !isValidKey(key) ||
324             normalizeKey(key) != key) {
325             throw PatriciaError("invalid file format");
326         }
327         const std::uint64_t value = readUnsigned(input, sizeof(std::uint64_t));
328         if (!trie.insert(key, value)) {
329             throw PatriciaError("invalid file format");
330         }
331     }
332     if (input.peek() != std::char_traits<char>::eof()) {
333         throw PatriciaError("invalid file format");
334     }
335     return trie;
336 }
337
338 void PatriciaTrie::saveToFile(const std::string& path) const {
339     std::ofstream file(path, std::ios::binary | std::ios::trunc);
340     if (!file) {
341         throw PatriciaError("cannot open file for writing");
342     }
343     save(file);
344     file.close();
345     if (!file) {
346         throw PatriciaError("failed to write file");
347     }
348 }
349
350 void PatriciaTrie::loadFromFile(const std::string& path) {
351     std::ifstream file(path, std::ios::binary);
352     if (!file) {
353         throw PatriciaError("cannot open file for reading");
354     }
355     PatriciaTrie loaded = load(file);
356     swap(loaded);
357 }

```

Listing 3: include/dictionary.hpp

```

1  #pragma once
2
3  #include <iosfwd>
4  #include <optional>
5  #include <string>
6  #include <string_view>
7
8  #include "patricia_trie.hpp"

```

```

9
10 class Dictionary {
11 public:
12     std::optional<std::string> execute(std::string_view line);
13
14 private:
15     std::string add(std::string_view key, std::string_view value);
16     std::string erase(std::string_view key);
17     std::string lookup(std::string_view key) const;
18     std::string runFileCommand(std::string_view command, const std::string& path);
19
20     PatriciaTrie trie_;
21 };
22
23 void runDictionary(std::istream& input, std::ostream& output);

```

Listing 4: src/dictionary.cpp

```

1 #include "dictionary.hpp"
2
3 #include <algorithm>
4 #include <charconv>
5 #include <cstdint>
6 #include <cstdint>
7 #include <istream>
8 #include <new>
9 #include <optional>
10 #include <ostream>
11 #include <string>
12 #include <string_view>
13 #include <system_error>
14 #include <vector>
15
16 #include "patricia_trie.hpp"
17
18 namespace {
19
20 bool isSpace(char symbol) {
21     return symbol == ' ' || symbol == '\t' || symbol == '\r' || symbol == '\n' || symbol == '\v' ||
22         symbol == '\f';
23 }
24
25 std::vector<std::string_view> splitTokens(std::string_view line) {
26     std::vector<std::string_view> tokens;
27     std::size_t position = 0;
28     while (position < line.size()) {
29         while (position < line.size() && isSpace(line[position])) {
30             ++position;
31         }
32         const std::size_t start = position;
33         while (position < line.size() && !isSpace(line[position])) {
34             ++position;
35         }
36         if (position > start) {
37             tokens.push_back(line.substr(start, position - start));
38         }
39     }
40     return tokens;
41 }
42
43 std::optional<std::uint64_t> parseValue(std::string_view text) {

```

```

44     const bool allDigits =
45         std::ranges::all_of(text, [](char symbol) { return symbol >= '0' && symbol <= '9'; });
46     if (text.empty() || !allDigits) {
47         return std::nullopt;
48     }
49     std::uint64_t value = 0;
50     const auto [end, error] = std::from_chars(text.data(), text.data() + text.size(), value);
51     if (error != std::errc{} || end != text.data() + text.size()) {
52         return std::nullopt;
53     }
54     return value;
55 }
56
57 constexpr std::string_view INVALID_COMMAND = "ERROR: invalid command";
58 constexpr std::string_view UNKNOWN_COMMAND = "ERROR: unknown command";
59 constexpr std::string_view INVALID_KEY = "ERROR: invalid key";
60 constexpr std::string_view INVALID_VALUE = "ERROR: invalid value";
61
62 }
63
64 std::optional<std::string> Dictionary::execute(std::string_view line) {
65     const std::vector<std::string_view> tokens = splitTokens(line);
66     if (tokens.empty()) {
67         return std::nullopt;
68     }
69     const std::string_view command = tokens.front();
70     if (command == "+") {
71         if (tokens.size() != 3) {
72             return std::string(INVALID_COMMAND);
73         }
74         return add(tokens[1], tokens[2]);
75     }
76     if (command == "-") {
77         if (tokens.size() != 2) {
78             return std::string(INVALID_COMMAND);
79         }
80         return erase(tokens[1]);
81     }
82     if (command == "!") {
83         if (tokens.size() < 2 || (tokens[1] != "Save" && tokens[1] != "Load")) {
84             return std::string(UNKNOWN_COMMAND);
85         }
86         if (tokens.size() != 3) {
87             return std::string(INVALID_COMMAND);
88         }
89         return runFileCommand(tokens[1], std::string(tokens[2]));
90     }
91     if (tokens.size() != 1) {
92         return std::string(UNKNOWN_COMMAND);
93     }
94     return lookup(command);
95 }
96
97 std::string Dictionary::add(std::string_view key, std::string_view value) {
98     if (!isValidKey(key)) {
99         return std::string(INVALID_KEY);
100     }
101     const std::optional<std::uint64_t> number = parseValue(value);
102     if (!number) {
103         return std::string(INVALID_VALUE);
104     }
105     return trie_.insert(normalizeKey(key), *number) ? "OK" : "Exist";

```

```

106 }
107
108 std::string Dictionary::erase(std::string_view key) {
109     if (!isValidKey(key)) {
110         return std::string(INVALID_KEY);
111     }
112     return trie_.remove(normalizeKey(key)) ? "OK" : "NoSuchWord";
113 }
114
115 std::string Dictionary::lookup(std::string_view key) const {
116     if (!isValidKey(key)) {
117         return std::string(INVALID_KEY);
118     }
119     const std::optional<std::uint64_t> value = trie_.find(normalizeKey(key));
120     if (!value) {
121         return "NoSuchWord";
122     }
123     return "OK: " + std::to_string(*value);
124 }
125
126 std::string Dictionary::runFileCommand(std::string_view command, const std::string& path) {
127     try {
128         if (command == "Save") {
129             trie_.saveToFile(path);
130         } else {
131             trie_.loadFromFile(path);
132         }
133     } catch (const PatriciaError& error) {
134         return std::string("ERROR: ") + error.what();
135     }
136     return "OK";
137 }
138
139 void runDictionary(std::istream& input, std::ostream& output) {
140     Dictionary dictionary;
141     std::string line;
142     while (std::getline(input, line)) {
143         std::optional<std::string> response;
144         try {
145             response = dictionary.execute(line);
146         } catch (const std::bad_alloc&) {
147             response = "ERROR: out of memory";
148         }
149         if (response) {
150             output << *response << '\n';
151         }
152     }
153 }

```

Listing 5: app/main.cpp

```
1 | #include <iostream>
2 |
3 | #include "dictionary.hpp"
4 |
5 | int main() {
6 |     std::ios::sync_with_stdio(false);
7 |     std::cin.tie(nullptr);
8 |
9 |     runDictionary(std::cin, std::cout);
10 |    return 0;
11 | }
```

3 Console

```
# make build PRESET=release
# build/release/lab2/lab2_main
+ hello 42
+ Hello 99
+ world 7
hello
WORLD
- hello
hello
OK
Exist
OK
OK: 42
OK: 7
OK
NoSuchWord
```

4 Performance test

1 Method

For every size N the generator `lab2/benchmark/generator.py` produces a set of unique keys of lowercase English letters with random length from 1 to 16 characters, using a fixed seed. It writes N insertion commands, then N lookups as plain `word` lines, then N removal commands to `lab2/benchmark/tests.txt`. Three operations are measured separately: insertion of N keys, lookup of all N keys and removal of all N keys. The program `lab2/benchmark/benchmark.cpp` compares PatriciaTrie with `std::map`, a balanced search tree from the STL with guaranteed $O(\log n)$ complexity for all operations.

The time of every operation is measured with `std::chrono::steady_clock`. Data generation and input/output are not included in the measurement.

2 Results

	PATRICIA, μs			std::map, μs		
N	Insert	Find	Remove	Insert	Find	Remove
1 000	348	208	465	214	161	303
10 000	4 731	3 246	8 264	4 251	2 405	3 711
100 000	86 469	75 838	100 506	87 293	73 691	81 991
500 000	731 836	695 895	764 450	766 442	748 168	785 446
1 000 000	1 842 835	2 022 130	2 038 648	1 953 836	1 985 049	2 322 171

Table 1: Operation time in microseconds (measured in Docker, gcc 14, Release build with -O3)

```
# make build PRESET=release
# python3 lab2/benchmark/generator.py 1000
# build/release/lab2/lab2_benchmark < lab2/benchmark/tests.txt
N=1000
PATRICIA   Insert=348us Find=208us Remove=465us
std::map   Insert=214us Find=161us Remove=303us
# python3 lab2/benchmark/generator.py 10000
# build/release/lab2/lab2_benchmark < lab2/benchmark/tests.txt
N=10000
PATRICIA   Insert=4731us Find=3246us Remove=8264us
std::map   Insert=4251us Find=2405us Remove=3711us
# python3 lab2/benchmark/generator.py 100000
```

```

# build/release/lab2/lab2_benchmark < lab2/benchmark/tests.txt
N=100000
PATRICIA   Insert=86469us Find=75838us Remove=100506us
std::map   Insert=87293us Find=73691us Remove=81991us
# python3 lab2/benchmark/generator.py 500000
# build/release/lab2/lab2_benchmark < lab2/benchmark/tests.txt
N=500000
PATRICIA   Insert=731836us Find=695895us Remove=764450us
std::map   Insert=766442us Find=748168us Remove=785446us
# python3 lab2/benchmark/generator.py 1000000
# build/release/lab2/lab2_benchmark < lab2/benchmark/tests.txt
N=1000000
PATRICIA   Insert=1842835us Find=2022130us Remove=2038648us
std::map   Insert=1953836us Find=1985049us Remove=2322171us

```

3 Analysis of the results

On small inputs ($N \leq 10\,000$) `std::map` is usually faster: the standard implementation is well optimized, while the overhead of PATRICIA for node allocation, bit checks and string storage does not pay off yet.

On the medium size ($N = 100\,000$) the results are close: insertion and lookup differ only slightly, while removal in PATRICIA remains more expensive because of the more complex restoration of links after a node is deleted.

On large sizes ($N \geq 500\,000$) PATRICIA starts to win: for $N = 500\,000$ all three operations are faster than in `std::map`, and for $N = 10^6$ PATRICIA is faster in insertion and removal. This agrees with the idea of a compressed trie: the path is determined by the critical bits of the keys rather than by a full lexicographic string comparison at every level of a balanced tree.

5 Work diary

- 10.05.2026: studied the theory of the PATRICIA tree, examined the mechanism of critical bits, back edges and the header node.
- 10.05.2026: implemented search, insertion and removal with case-insensitive keys and all main removal cases.
- 10.05.2026: added binary serialization of the dictionary, checked atomic loading through a temporary tree.
- 10.05.2026: wrote the generator and the benchmark, compared PATRICIA with `std::map`, prepared the results and conclusions.

6 Conclusions

In this laboratory work for the course “Discrete Analysis” I studied the structure and the implementation of the PATRICIA tree. I learned the mechanism of critical bits and back edges and implemented insertion, removal and search correctly with all boundary cases (removal of a leaf, removal of an internal node, a single node in the tree). I implemented binary serialization with format validation and atomic loading.

The benchmark against `std::map` showed that the overhead of PATRICIA does not pay off on small dictionaries, while for $N \geq 500\,000$ keys PATRICIA becomes faster in most operations, because the descent compares single critical bits instead of whole strings at every level. Removal remains the most expensive operation because of the restoration of back edges.

References

- [1] Cormen T. H., Leiserson C. E., Rivest R. L., Stein C. *Introduction to Algorithms, 2nd edition*. — MIT Press, 2001.
- [2] Morrison D. R. *PATRICIA — Practical Algorithm To Retrieve Information Coded In Alphanumeric*. Journal of the ACM, 15(4):514–534, 1968.
- [3] Sedgewick R. *Algorithms in C++, Parts 1–4: Fundamentals, Data Structures, Sorting, Searching, 3rd edition*. — Addison-Wesley, 1998.